

# The AI Agent Breakdown: LLM Brain vs. Harness System

A real walkthrough of the two parts inside every AI agent and how to read your own setup like an engineer.

## Why I made this

Most people using AI agents in their business only understand half of what is running. They know the model is smart, but they have no idea why an agent sometimes fails even when the model reasoned correctly. The reason is almost always the other half of the system: the harness.

An AI agent is not one thing. It is two systems working together: the LLM, the brain that reasons through steps, and the harness, the system wrapped around it that actually makes things happen. Once you see both halves, your agent setup stops being a black box and starts being something you can debug.

01

## How it actually works

1. The LLM reads the current context and decides what to do next. When it needs to take an action it cannot do itself, like sending an email or checking a file, it outputs a structured tool call instead of a plain-text answer.
2. The harness reads that structured call and runs the real action. This is the part with actual permissions and connections: it manages memory, chat history, files, contacts, and connections to external tools like Gmail or an image and video generator.
3. The harness drops the result of that action back into the model's context. The model reads it and decides the next move. This decide, execute, observe loop repeats until the task is finished.

02

## The one thing almost everyone gets wrong

People troubleshoot a misbehaving agent by blaming the model. Most of the time the model reasoned fine. The failure is in the harness: a missing tool connection, memory that got lost between steps, or a permission the harness never had in the first place.

### WATCH OUT

When an agent seems to forget what it was doing, check the harness first, not the model. Memory and chat history live in the harness. If the harness drops context between steps, the model has no way to recover it on its own.

## AGENT-LOOP.TXT

1. LLM decides
  - > reads current context
  - > outputs a structured tool call (not plain text)
2. Harness executes
  - > runs the real action: API call, file write, script, subagent spawn
  - > has the actual permissions and connections
3. Harness observes
  - > drops the tool's result back into the model's context
4. LLM decides again
  - > reads the new context
  - > loop repeats until the task is done

## WHERE THIS SHOWS UP IN CLAUDE CODE

- Claude Code docs: MCP servers  
<https://docs.claude.com/en/docs/claude-code/mcp>
- Claude Code docs: Subagents  
<https://docs.claude.com/en/docs/claude-code/sub-agents>

## Map your own agent setup in Claude Code or Codex

1. Open the project you want to map and list every harness-level piece you are actually using: MCP servers, Bash access, file read/write tools, and any subagents you spawn with the Agent tool.
2. In Claude Code, these live under `~/.claude/skills/` for installed skills and in your project's `.mcp.json` or settings for MCP connections. In Codex, the equivalent lives in your project folder or `.codex/skills/`, with a pointer in `AGENTS.md`.
3. Separate your notes into two columns: 'LLM' (which model, which version you are running) and 'Harness' (every tool, memory store, and delegation path it can reach). This is the same split the video walks through, just applied to your own setup.
4. If any tool needs an API key, ask Claude to add it to your shell profile (`~/.zshrc` or `~/.bashrc`), then open a new terminal so the key loads before you test anything.
5. Run one simple task that should touch at least one tool call, for example asking the agent to read a file and summarize it. Watch the transcript: you should see a structured tool call, a result dropped back in, and the model responding to that result. That is the loop from this guide happening in front of you.
6. A successful first run looks like: the model names the tool it is using, the harness returns a real result (not an error), and the model's next message directly references that result instead of guessing.

 QUICK TIPS

- If you cannot tell which part failed, read the raw tool call and its result first. A correct tool call with a bad result is a harness problem. A missing or malformed tool call is a model-reasoning problem.
- Keep a running list of every tool your harness can reach. It is the fastest way to spot why an agent 'can't' do something: it usually just was never given the connection.

## 04

### The commands I actually run

#### MAP WHAT MY HARNESS CAN REACH

List every tool, MCP server, and subagent type available to you right now in this project. For each one, tell me what it connects to and what permissions it has.

#### DEBUG A FAILED AGENT RUN

Walk through the last tool call you made step by step: what you decided, what the harness returned, and what you did with that result. Tell me exactly where the chain broke.

#### CHECK WHAT THE HARNESS IS STORING IN MEMORY

Summarize what you currently remember about this project from earlier in this session versus what you would need to re-read from files if your memory were cleared right now.

#### PLAN A DELEGATION TASK

This task has [N] independent parts: [LIST PARTS]. Tell me which parts could run as separate subagent instances in parallel versus which ones depend on each other and must run in order.

### When it breaks

| ERROR                                                    | FIX                                                                                                                                    |
|----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Agent 'forgets' something from earlier in the task       | Harness memory was dropped between steps. Check how chat history is being passed back into context, not the model's reasoning.         |
| Model keeps saying it can't access a tool it should have | The harness never connected that tool for this session. Check your MCP server list or tool permissions, not the prompt.                |
| Agent repeats the same failed action in a loop           | The harness is returning an error the model doesn't understand. Read the raw tool result; the model is reacting correctly to bad data. |
| Parallel subagent task finishes with missing results     | One subagent instance likely failed silently. Check each delegated task's output individually instead of only the combined result.     |
| Tool call never happens even though you asked for it     | The model chose not to call it, which is a reasoning issue. Rephrase the request to make the needed action explicit.                   |

## Ready to go deeper?

This playbook is the short version. Inside the hub you get the full build walkthroughs, the prompt library, and the systems I use to ship this in a weekend.

**Join the Build →**

<https://hub.digicuratoragency.com/join>